Why AI Agents Still Need You: Findings from Developer-Agent Collaborations in the Wild

Nikhil Anand

April 6, 2026

This paper investigates how software developers collaborate with AI-based software engineering agents (SWE agents) in real-world settings, highlighting both the promise and limitations of such tools. It begins by noting that while SWE agents can autonomously solve structured benchmark problems, they struggle with complex, ambiguous, real-world issues due to missing contextual and tacit knowledge. To address this gap, the authors conduct an empirical study of 19 developers using an in-IDE agent (Cursor) to resolve 33 real issues from repositories they were familiar with. The study finds that collaboration is essential: developers who iteratively worked with the agent and actively guided it were more successful than those who relied on one-shot automation, achieving an overall success rate of about 55%.

The paper further highlights key patterns and challenges in developer-agent collaboration. Developers often treat agents as assistants rather than equal collaborators, delegating code generation while retaining responsibility for debugging, testing, and validation. However, several barriers hinder effective collaboration, including lack of trust, agent overconfidence, sycophantic behavior (agreeing too readily with users), and unsolicited or excessive actions. The findings suggest that successful collaboration requires active participation from both developers and agents across all stages of software development. Rather than acting as passive tools, agents should engage in meaningful dialogue, challenge assumptions, and support iterative workflows, ultimately positioning human–AI collaboration—not full automation—as the most effective paradigm for real-world software engineering tasks.

1 Motivation

The motivation for this paper stems from the rapid adoption of AI tools in software development and the emergence of more advanced **software engineering agents (SWE agents)** that go beyond simple code completion. Developers increasingly rely on generative AI systems to **boost productivity, generate code, and assist with problem-solving**, as evidenced by widespread usage of tools such as chat-based assistants and inline code generators [1]. More recently, SWE agents have been introduced as a new paradigm capable of **autonomously performing complex development tasks**, including writing code, executing commands, and iteratively refining outputs based on feedback [2]. While these agents have demonstrated strong performance on controlled benchmarks such as SWE-Bench [3], their effectiveness in real-world development remains uncertain.

A key motivation for this work is the **gap between benchmark success and real-world applicability**. Despite promising results in automated settings, SWE agents struggle with **complex, ambiguous issues, lack of tacit knowledge, and incomplete access to real development environments** [4, 5]. These limitations suggest that fully autonomous problem-solving is insufficient, motivating the need for **human-in-the-loop collaboration** where developers and agents share responsibilities across tasks such as understanding codebases, debugging, and testing. However, prior research has largely focused on either non-agentic tools (e.g., code completion) or fully autonomous agents, leaving a **lack of empirical understanding of how developers interact with SWE agents in practice** [2].

Another motivation arises from known challenges in human-AI interaction during software development. Previous studies show that developers often face difficulties in **trusting AI outputs, understanding generated code, communicating context effectively, and debugging AI-generated solutions**

[6, 7]. These challenges may be further amplified in SWE agents, which have **greater autonomy and decision-making capabilities**, potentially increasing cognitive load and complicating collaboration. At the same time, insights from software engineering research emphasize that effective collaboration depends on **communication, shared understanding, and tacit knowledge exchange**, which are difficult for AI systems to replicate [5, 8].

Together, these gaps motivate the central aim of the paper: to **empirically study developer-agent collaboration in real-world settings**, understand how developers interact with SWE agents, identify barriers to effective collaboration, and uncover factors that influence success. By doing so, the work seeks to inform the design of future SWE agents that better support **interactive, collaborative workflows rather than purely autonomous execution**, ultimately improving both developer productivity and the effectiveness of human–AI collaboration in software engineering.

2 Methodology

Study Design and Objective The study aims to observe how developers collaborate with SWE agents in practice, focusing on three research questions related to collaboration patterns, barriers, and success factors. To ensure ecological validity, the authors designed the study to take place **"in the wild"**, where participants worked on real issues in familiar repositories rather than synthetic tasks [2]. This design ensures that findings reflect **authentic developer behavior and real-world constraints**, rather than idealized benchmark scenarios.

Tool Selection The authors evaluated multiple SWE agents across a spectrum of autonomy and collaboration capabilities, ranging from fully autonomous systems (e.g., AutoCodeRover) to interactive, IDE-based agents such as Cursor and VSCode Agent Mode. They selected **Cursor Agent** because it supports **high levels of human–AI collaboration**, including features like real-time interaction, code editing, execution, and contextual awareness (e.g., using open files as input) [9]. The final setup used Cursor v0.47 with Claude 3.5 Sonnet, ensuring a consistent interaction environment for all participants.

Task Selection To simulate realistic development work, participants were asked to solve **open issues (bugs or feature requests)** in repositories they had previously contributed to. The tasks were carefully selected based on the following criteria:

- **Representative of real-world development tasks**
- **Motivating for participants to solve thoroughly**
- **Completable within approximately one hour**
- **Non-proprietary (open-source repositories)**

Additionally, repositories were filtered to ensure quality and relevance: they had to be **actively maintained, sufficiently large (¿50 source files), and widely used (500+ GitHub stars)** [10, 11]. Participants primarily selected their own issues, with guidance provided to ensure diversity in task type (e.g., bugs vs features) and difficulty.

Participants The study involved **19 professional developers** recruited from contributors to selected repositories. Participants represented diverse backgrounds in terms of:

- **Experience levels** (ranging from 0–2 years to 16+ years)
- **Roles** (engineers, senior engineers, managers)
- **Geographic regions and demographics**

Most participants had prior experience with AI tools such as GitHub Copilot, though only a few had used SWE agents like Cursor before. This ensured that results reflect **early-stage interaction patterns with agentic tools**.

Study Protocol Each session lasted approximately **60 minutes** and was conducted via remote collaboration. The protocol consisted of:

1. **Pre-study questionnaire and consent**
2. **Tutorial session** introducing Cursor Agent and its features
3. **Main task execution**, where participants:
    - Worked on selected issues
    - Were encouraged to **think aloud**
    - Used the agent for most tasks but could make manual edits
4. **Post-study questionnaire** capturing perceptions of the tool

Participants interacted with the agent on a pre-configured system to avoid setup overhead. They were instructed to aim for a solution they would be confident submitting as a pull request, ensuring **high-quality engagement with the task**.

Data Collection The study collected multiple data sources to analyze collaboration:

- **Session recordings (video, audio, screen)** to capture interaction behavior
- **Participant action logs**, categorized using an adapted CUPS taxonomy [12]
- **Chat interaction trajectories**, capturing prompts, context provided, and agent responses
- **Pre- and post-study questionnaires** assessing usability, perception, and experience

The researchers developed structured codebooks for both participant actions and chat trajectories using open coding and iterative refinement. Inter-rater reliability was high (Cohen's κ of 0.92 and 0.89), ensuring **robust qualitative analysis** [13].

Analysis Approach The analysis focused on **qualitative patterns rather than quantitative metrics**. The authors:

- Analyzed **presence and sequence of actions** rather than duration
- Examined **prompt-level and issue-level interactions**
- Identified recurring behaviors, strategies, and barriers across participants

Given the exploratory nature and small sample size, the study avoided inferential statistics and instead presented **descriptive insights supported by observed trends** [14]. Member checking was performed by sharing findings with participants to validate interpretations.

3 Key Findings

The study finds that successful use of SWE agents is fundamentally driven by **active, iterative collaboration rather than one-shot automation**. Developers who continuously interacted with the agent—refining prompts, providing additional context, and verifying intermediate outputs—were significantly more likely to complete tasks successfully. In contrast, those who treated the agent as a fully autonomous system often encountered failures due to incomplete understanding of the problem or incorrect assumptions. This highlights that **human guidance remains essential**, and effective workflows resemble a back-and-forth problem-solving process rather than delegation.

Another key finding is that developers tend to treat SWE agents as **assistants rather than equal collaborators**. Participants primarily relied on the agent for **code generation and initial exploration**, while retaining responsibility for higher-level reasoning tasks such as debugging, validation, and decision-making. This division of labor indicates that while agents can accelerate development, developers still play a critical role in ensuring correctness and aligning solutions with project-specific requirements. It

also suggests that current agents lack sufficient capability to independently manage complex, end-to-end development workflows.

The study also uncovers several behavioral patterns that influence success. Developers who provided **clear, structured, and context-rich prompts** achieved better outcomes, as the agent was more likely to generate relevant and accurate solutions. Additionally, effective users frequently engaged in **verification practices**, such as reviewing generated code, running tests, and cross-checking logic. These behaviors demonstrate that **prompting and validation are core skills** when working with SWE agents, shaping both the quality of outputs and the overall efficiency of the interaction.

Despite their potential, SWE agents exhibit notable limitations that hinder collaboration. Participants reported issues such as **overconfidence in incorrect outputs**, **lack of transparency in reasoning**, and **difficulty incorporating implicit or tacit knowledge about the codebase**. The agents also displayed tendencies toward **sycophantic behavior**, often agreeing with user suggestions even when they were flawed, and occasionally performing **unsolicited or excessive actions** that disrupted workflows. These challenges contribute to reduced trust and require developers to remain vigilant throughout the interaction.

Finally, the findings emphasize that effective developer–agent collaboration requires both parties to actively contribute across the software development lifecycle. The most successful interactions occurred when developers and agents engaged in **continuous dialogue**, shared context, and iteratively refined solutions. This leads to the broader conclusion that **human–AI collaboration, rather than full automation, is the most viable paradigm** for real-world software engineering. Future systems must therefore focus on improving interactivity, transparency, and alignment with developer workflows to fully realize the benefits of SWE agents.

4 Implications

The findings of this study have important implications for the design of future SWE agents. First, they highlight that current systems should move away from a purely automation-centric paradigm and instead prioritize **interactive, human-in-the-loop collaboration**. Since successful outcomes depend heavily on iterative engagement, future agents must be designed to **support continuous dialogue, ask clarifying questions, and adapt to evolving user input**. This suggests that conversational capabilities, contextual awareness, and responsiveness are not optional features but **core requirements for effective developer–agent interaction**.

Another key implication is the need to improve **transparency and trustworthiness** of agent behavior. Developers often struggled with overconfident or opaque outputs, which forced them to spend additional effort verifying results. To address this, agents should provide **explanations of their reasoning, highlight uncertainties, and surface assumptions** made during code generation or modification. Enhancing transparency can reduce cognitive load and help developers make more informed decisions, ultimately improving both efficiency and trust in AI-assisted workflows.

The study also suggests that SWE agents should better support **developer workflows across the entire software development lifecycle**, rather than focusing narrowly on code generation. Developers naturally took responsibility for tasks such as debugging, testing, and validation, indicating that agents currently lack holistic support. Future systems should therefore integrate capabilities for **test generation, error diagnosis, code navigation, and environment understanding**, enabling more seamless end-to-end collaboration and reducing fragmentation in the development process.

Another implication lies in the importance of **user skill and interaction strategies**. Since outcomes depend on how effectively developers communicate with the agent, there is a growing need for **guidelines, training, and tooling that support better prompting and interaction practices**. This could include structured prompt templates, interactive debugging aids, and feedback mechanisms that guide users toward more effective collaboration patterns. Supporting these skills can help bridge the gap between novice and expert users of SWE agents.

Finally, the findings emphasize the importance of designing agents that behave as **proactive collaborators rather than passive assistants**. Instead of simply following instructions, agents should be

capable of **challenging incorrect assumptions, avoiding sycophantic agreement, and making context-aware suggestions**. By fostering a more balanced partnership, future SWE agents can contribute more meaningfully to problem-solving and reduce the burden on developers, ultimately leading to more robust and effective human–AI collaboration in software engineering.

5 Threats to Validity

The study is subject to several threats to validity arising from its design and scope. One key limitation is the **small sample size and participant selection**. With only a limited number of developers involved, the findings may not generalize to the broader population of software engineers, especially those with different levels of expertise or from different domains. Additionally, participants were recruited from contributors to specific open-source repositories, which may introduce **selection bias**, as these developers could be more experienced, motivated, or familiar with collaborative tools than average practitioners.

Another threat concerns the **task and environment constraints**. Although the study attempts to simulate real-world conditions by using actual issues from familiar repositories, the tasks were still limited to those that could be completed within a short time frame (around one hour). This may not accurately reflect the complexity, scale, and long-term dependencies of real software engineering work. Furthermore, the use of a **single tool (Cursor Agent) and a fixed model configuration** restricts the generalizability of the findings, as different SWE agents or models may exhibit different behaviors and collaboration dynamics.

There are also potential threats related to **data collection and analysis methods**. The qualitative nature of the study relies on manual coding of participant actions and interactions, which may introduce **researcher bias** despite efforts to ensure consistency through structured codebooks and inter-rater agreement. Additionally, the think-aloud protocol and recorded sessions may have influenced participant behavior, leading to **observation bias** where developers act differently than they would in their natural workflow.

Finally, the study may be affected by **learning and novelty effects**. Since many participants had limited prior experience with SWE agents, their interactions may reflect early-stage exploration rather than mature usage patterns. This could result in underestimating the potential effectiveness of such tools once users become more familiar with them. Moreover, the rapidly evolving nature of AI systems means that the findings could become outdated as models and tools improve, posing a **temporal validity threat** to the long-term applicability of the results.

References

[1] *Stack Overflow Developer Survey 2024*. 2024.

[2] J. Liu et al. "Large Language Model-Based Agents for Software Engineering: A Survey". In: (2024).

[3] C. E. Jimenez et al. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" In: (2024).

[4] S. Miserendino et al. "SWE-Lancer: Can Frontier LLMs Earn $1 Million from Real-World Freelance Software Engineering?" In: (2024).

[5] I. Rus and M. Lindvall. "Knowledge Management in Software Engineering". In: *IEEE Software* 19.3 (2002), pp. 26–38.

[6] S. Barke, M. B. James, and N. Polikarpova. "Grounded Copilot: How Programmers Interact with Code-Generating Models". In: *Proceedings of the ACM on Programming Languages*. 2023.

[7] P. Vaithilingam, T. Zhang, and E. L. Glassman. "Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models". In: *CHI Conference on Human Factors in Computing Systems*. 2022.

[8] M. Goncalves, C. Souza, and V. Gonzalez. "Collaboration, Information Seeking and Communication: An Observational Study of Software Developers' Work Practices". In: *Journal of Universal Computer Science* 17 (2011), pp. 1913–1930.

[9] *Cursor Agent Documentation*. https://docs.cursor.com/chat/agent. 2025.

[10] J. Zhang et al. “Inside Bug Report Templates: An Empirical Study on Bug Report Templates in Open-Source Software”. In: 2024.
[11] Y. Sens et al. “A Large-Scale Study of Model Integration in ML-Enabled Software Systems”. In: (2025).
[12] H. Mozannar et al. “Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming”. In: 2024.
[13] J. R. Landis and G. G. Koch. “The Measurement of Observer Agreement for Categorical Data”. In: *Biometrics* (1977).
[14] S. L. Motulsky. “Is Member Checking the Gold Standard of Quality in Qualitative Research?” In: *Qualitative Psychology* (2021).